

The background of the slide is a stylized map with grey streets and green parks. A large yellow semi-circle is positioned behind the title, and a yellow triangle points downwards below the title bar. Dashed black lines are scattered across the map. The title 'SafeNav' is centered in a green box.

SafeNav

Adlin Hamdan, Fatma Azzab, Nithila Balaji, Yuhan Wang, Zhenya Ratushko

Table of Contents

01

Background



02

Build Process

03

Demonstration

04

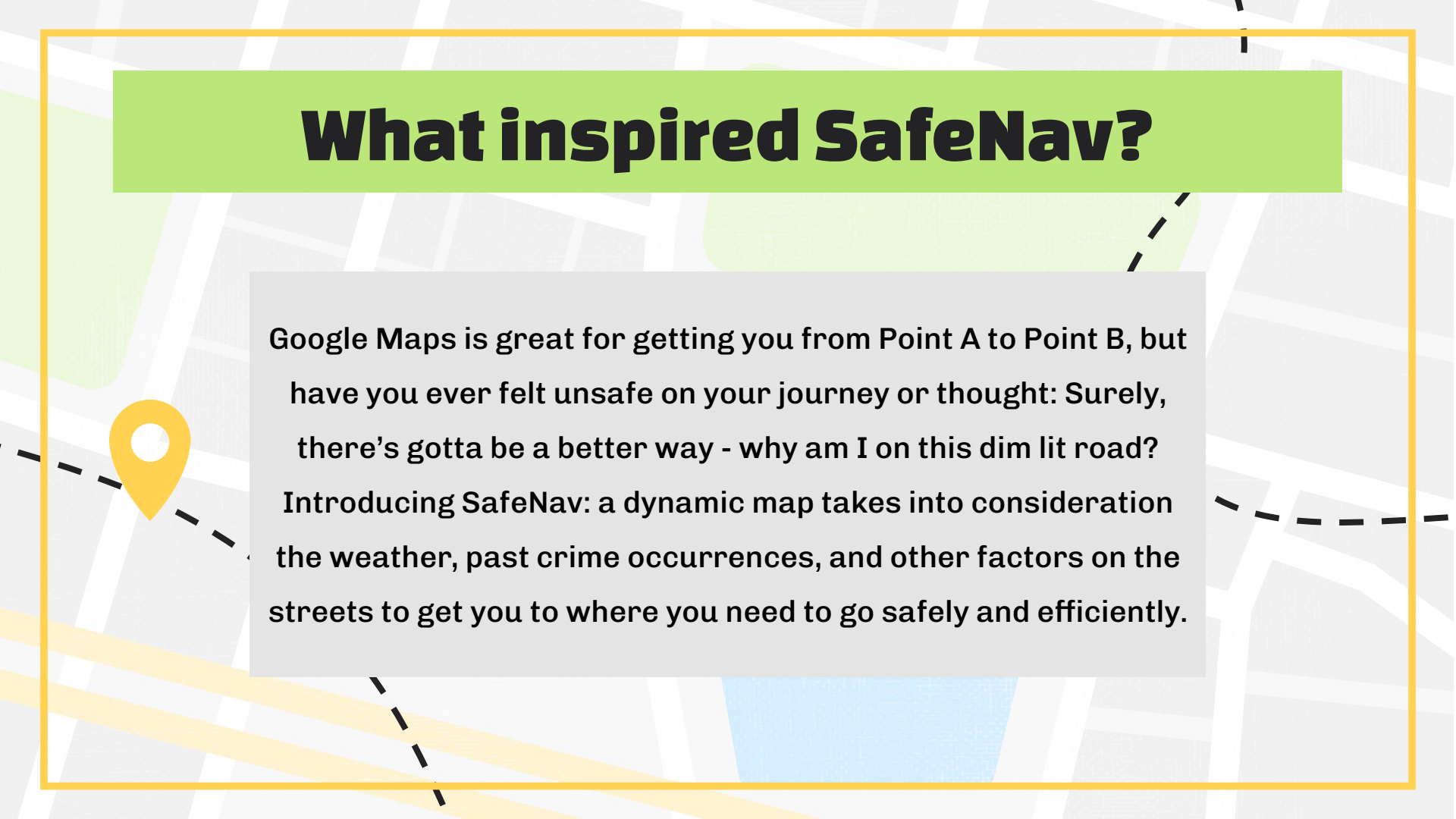
Takeaways + Future Expansion

The background is a stylized map with a grid of light gray streets. A large, semi-transparent green rectangle is positioned in the upper left. A blue number '01' is centered in the upper half. A yellow location pin is in the upper right. A dashed black line starts from the left edge, passes through the green rectangle, and ends near the location pin. Another dashed black line starts from the bottom right and extends towards the center.

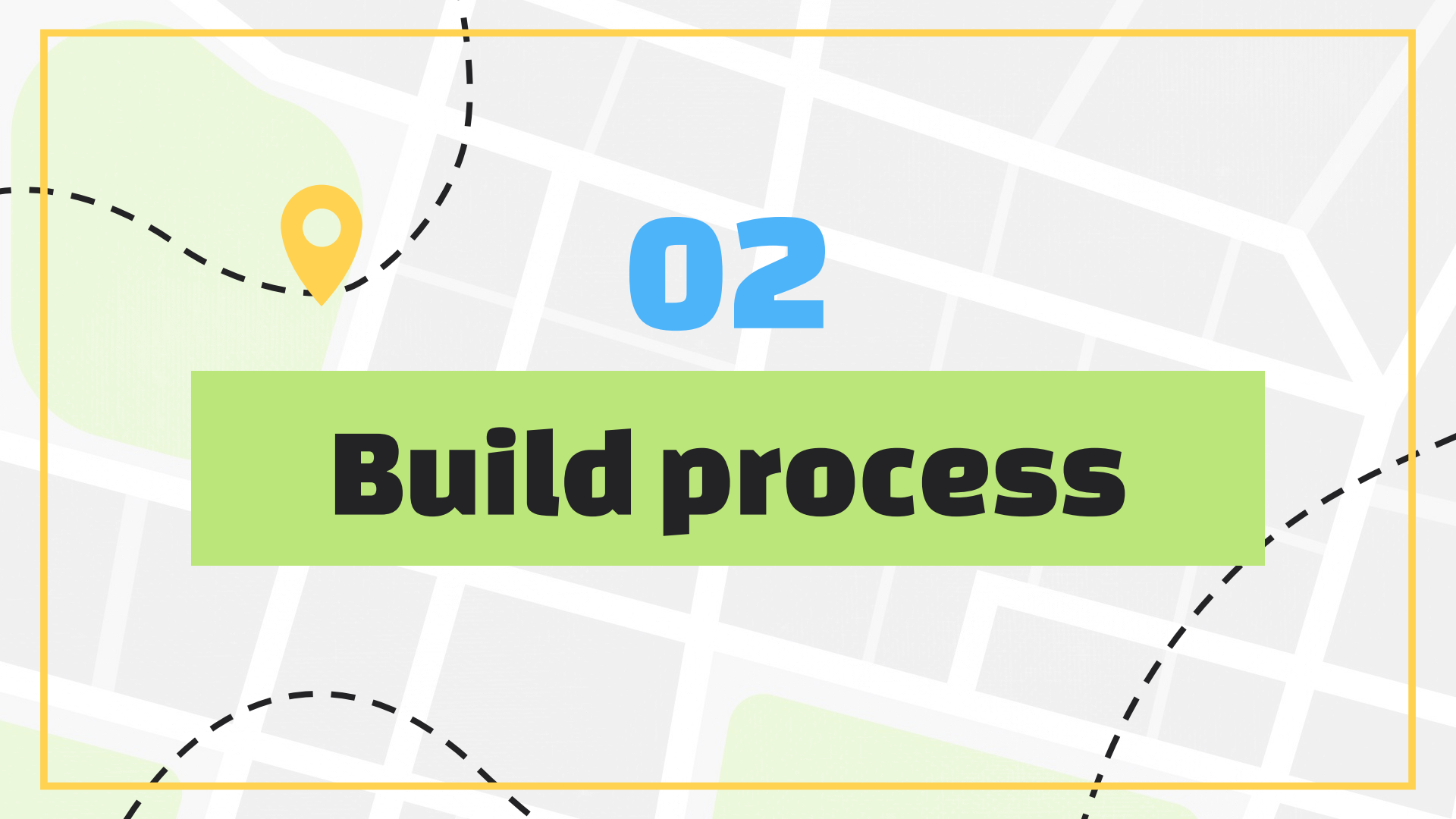
01

Background

What inspired SafeNav?



Google Maps is great for getting you from Point A to Point B, but have you ever felt unsafe on your journey or thought: Surely, there's gotta be a better way - why am I on this dim lit road? Introducing SafeNav: a dynamic map takes into consideration the weather, past crime occurrences, and other factors on the streets to get you to where you need to go safely and efficiently.



02

Build process

Collecting and wrangling data

- Extracting real-time weather data for Madison, WI using an API call to weather.gov
 - Calculating weather factors
 - > determining best routes
- Building a web-scraping function to scrape data from the City of Madison police report database (notable incidents from 01/01/24 - 01/01/25)
 - Grouping by low/medium/high severity and time of day
 - > determining best routes

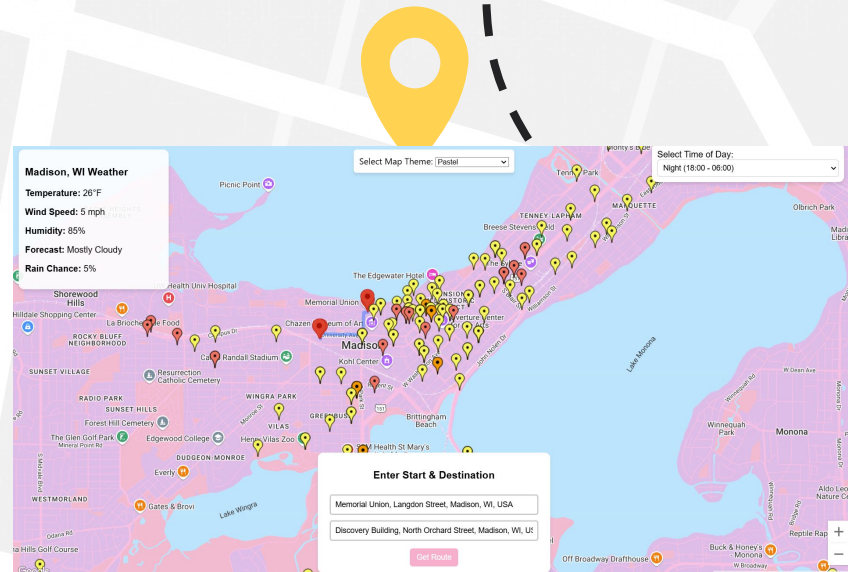
Released	Incident	Case Number	Address	Released By	Time of Incident
02/19/2024	Weapons Violation	2024-00067954	7500 block Westward Way	P.O. Korrie Rondorf	20:58:00
02/20/2024	Robbery	2024-00071935	2900 block Coho St	P.O. Korrie Rondorf	20:02:00
02/22/2024	Arrested Person	2024-00073417	500 block State St	P.O. Korrie Rondorf	19:05:00
02/22/2024	Arrested Person	2024-00075091	2500 block Fich Hatchery Rd	P.O. Korrie Rondorf	20:37:00
02/22/2024	Weapons Violation	2024-00072439	1400 block McKenna Blvd	P.O. Korrie Rondorf	08:23:00
02/22/2024	Robbery	2024-00074134	5700 block Raymond Rd	P.O. Korrie Rondorf	09:27:00
02/22/2024	Disturbance	2024-00075412	100 block Dayton St	P.O. Korrie Rondorf	02:32:00
02/22/2024	Disturbance	2024-00075412	100 block Dayton St	P.O. Korrie Rondorf	02:32:00
02/23/2024	Missing Adult	2024-076800	1600 Northport Dr	P.O. Anthony Vogel	20:13:00
02/23/2024	Sexual Assault	2005-124598	2000 Block Eastwood Dr.	P.O. Anthony Vogel	20:13:00
02/23/2024	Disturbance	2024-076428	600 E Washington Ave	P.O. Anthony Vogel	16:10:00
02/27/2024	Disturbance	2024-81200	400 block W. Gilman St.	PIO Stephanie Fryer	20:20:00
03/01/2024	Battery	2024-87495	1800 block Loftsgordon Ave.	PIO Stephanie Fryer	23:43:00
03/01/2024	Fight (In Progress)	2024-00086656	2222 E. Washington Ave.	PIO Stephanie Fryer	12:30:00
03/01/2024	Traffic Incident/Road Rage	2024-86393	Tokay Blvd. at S. Whitney Way	PIO Stephanie Fryer	09:18:00
03/01/2024	Weapons Violation	2024-085825	200 block of State St.	PIO Stephanie Fryer	20:30:00
03/01/2024	Residential Burglary	2024-900620	600 block W. Main St.	PIO Stephanie Fryer	14:15:00
03/01/2024	Theft	2024-85843	State St. at Johnson St.	PIO Stephanie Fryer	20:15:00

sample police_reports.csv

Creating the React website

Integrates the Google Maps API, real-time weather data, and crime-mapping

- **Features**
 - Inclusive theme selection
 - Real-time weather updates
 - Routing
 - Crime awareness
- **Development**
 - Built with React for an interactive UI
 - Used google maps api for mapping/routing



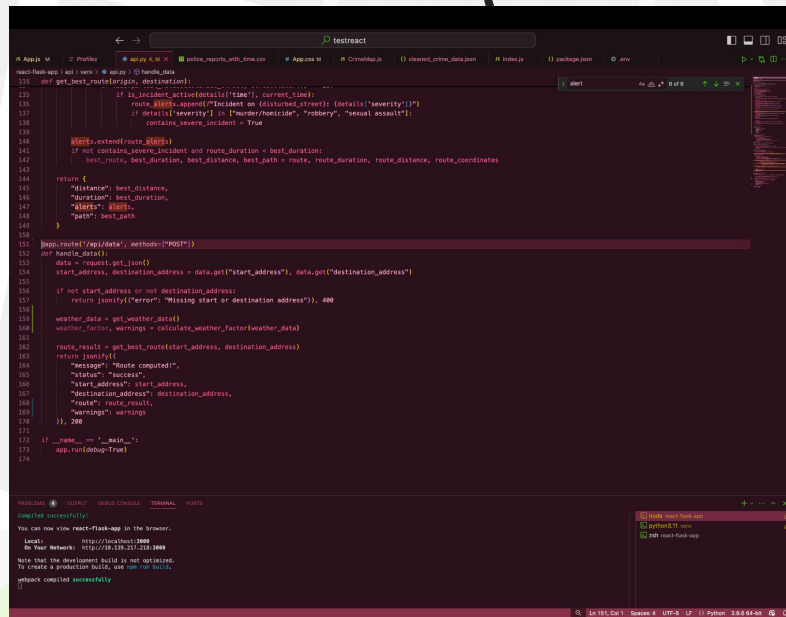
Linking it all with Flask

Set up Flask

- Create a Flask app with Flask and Flask-CORS to handle requests from React.
- Define API routes (@app.route) to handle GET and POST requests.
- Process data, such as user input or route calculations, and return JSON responses.
- Run the Flask app on a different port, e.g., 5000.

Connect React to Flask

- Use fetch() or axios in React to send requests to Flask's API.
- Use useEffect to fetch data on component mount and useState to store responses.
- Send user inputs (e.g., locations) from React via POST requests to Flask.
- Display Flask's response data (e.g., route info, weather, crime reports) in React components.



The screenshot shows a code editor with a Flask application. The code defines a Flask app with two routes: a GET route for '/api/data' and a POST route for '/api/data'. The GET route returns a JSON response with route information. The POST route processes user input and returns a JSON response with route information, weather data, and crime reports. The terminal output shows the Flask app running successfully on port 5000.

```
111 def get_best_route(origin, destination):
112     if is_incident_active(details['time'], current_time):
113         route_details.append({"incident on (disturbed street)": details['severity']})
114         if details['severity'] in ["murder/homicide", "robbery", "sexual assault"]:
115             contains_severe_incident = True
116     route_details.extend(route_details)
117     if not contains_severe_incident and route_duration < best_duration:
118         best_route, best_duration, best_distance, route_coordinates = route_details
119     return {
120         "message": best_distance,
121         "duration": best_duration,
122         "start_address": start_address,
123         "end_address": destination_address,
124         "route": best_path
125     }
126
127 @app.route("/api/data", methods=["POST"])
128 def handle_data():
129     data = request.get_json()
130     start_address, destination_address = data.get("start_address"), data.get("destination_address")
131
132     if not start_address or not destination_address:
133         return jsonify({"error": "Missing start or destination address"}), 400
134
135     weather_data = get_weather_data()
136     weather_factor, warnings = calculate_weather_factor(weather_data)
137
138     route_details = get_best_route(start_address, destination_address)
139     return jsonify({
140         "message": "Route computed!",
141         "status": "success",
142         "start_address": start_address,
143         "destination_address": destination_address,
144         "route": route_details,
145         "warnings": warnings
146     })
147
148 if __name__ == '__main__':
149     app.run(debug=True)
```

Compiled successfully!

You can now view react-flask-app in the browser.

Local: <http://localhost:3000>
On Your Network: <http://192.168.1.100:3000>

Note that the development build is not optimized.
To create a production build, use `npm run build`.

webpack compiled successfully



03

Demonstration

SafeNav in action

Embedded video

The image features a stylized map background with a grid of streets. A large, light gray rectangle is centered on the map, and it is crossed out with a large red 'X'. Inside this rectangle, the text 'Embedded video' is written in black. To the right of the rectangle, there is a dashed black line that curves upwards and then downwards, ending in a yellow location pin icon. The entire scene is framed by a thick orange border.



04



Takeaways + Future Ideas

SafeNav 2.0

- Add a dynamic element to the police reports data + have it update in real time
- Implement other modes of transportation (ex. cycling, driving)
- Incorporate filters
 - #1: No alert if low crime happened > 3 months ago, medium > 6 months, severe > 1 year ago
 - #2: Toggle between types of crime displayed
- Introduce a crowd-sourcing element: allow user input as to whether or not they are feeling unsafe and use ML predictive models to assess validity and use to guide further routes



The background is a stylized map with light gray streets and green areas representing parks. A dashed black line curves across the top left and bottom right. A solid green horizontal banner is centered across the middle of the image.

THANK YOU!